

Engenharia de Software II

André L. D. Rossi

Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP)
Faculdade de Ciências (FC) / Departamento de Computação (DCo)
Bauru, SP - Brasil

Conceitos de Projeto

O que é projeto?

- Projeto é o que todo engenheiro quer fazer

O que é projeto?

- Projeto é o que todo engenheiro quer fazer
- Modelo de projeto: “Como fazer?”

O que é projeto?

- Projeto é o que todo engenheiro quer fazer
- Modelo de projeto: “Como fazer?”
- Cria uma representação ou modelo de software

O que é projeto?

- Projeto é o que todo engenheiro quer fazer
- Modelo de projeto: “Como fazer?”
- Cria uma representação ou modelo de software
- Fornece detalhes:
 - Arquitetura de software
 - Estruturas de dados
 - Interfaces
 - Componentes

O que é Projeto?

Segundo Mitchell Kapor (1990) no seu Manifesto do Projeto (*Design*) de Software:

O que é Projeto?

Segundo Mitchell Kapor (1990) no seu Manifesto do Projeto (*Design*) de Software:

“It’s where you stand with a foot in two worlds—the world of technology and the world of people and human purposes—and you try to bring the two together.”

Por que elaborar um projeto?

- Permite “modelar” o sistema (produto) antes de ser codificado (construído)

Por que elaborar um projeto?

- Permite “modelar” o sistema (produto) antes de ser codificado (construído)
 - Etapa em que a qualidade é estabelecida

Por que elaborar um projeto?

Atributos de qualidade – **FURPS**:

- Funcionalidade (Functionality): características e capacidades do software

Por que elaborar um projeto?

Atributos de qualidade – **FURPS**:

- Funcionalidade (Functionality): características e capacidades do software
- Usabilidade (Usability) : fatores humanos, estética, consistência e documentação

Por que elaborar um projeto?

Atributos de qualidade – **FURPS**:

- Funcionalidade (Functionality): características e capacidades do software
- Usabilidade (Usability) : fatores humanos, estética, consistência e documentação
- Confiabilidade (Reliability) : frequência (MTTF) e gravidade das falhas

Por que elaborar um projeto?

Atributos de qualidade – **FURPS**:

- Funcionalidade (Functionality): características e capacidades do software
- Usabilidade (Usability) : fatores humanos, estética, consistência e documentação
- Confiabilidade (Reliability) : frequência (MTTF) e gravidade das falhas
- Desempenho (Performance) : eficiência e eficácia

Por que elaborar um projeto?

Atributos de qualidade – **FURPS**:

- Funcionalidade (Functionality): características e capacidades do software
- Usabilidade (Usability) : fatores humanos, estética, consistência e documentação
- Confiabilidade (Reliability) : frequência (MTTF) e gravidade das falhas
- Desempenho (Performance) : eficiência e eficácia
- Facilidade de Suporte (Supportability) : facilidade de manutenção (estender, adaptar, reparar, ...)

Por que elaborar um projeto?

Práticas de projeto:

Por que elaborar um projeto?

Práticas de projeto:

- **Diversificação:** identificar possíveis alternativas de projeto

Por que elaborar um projeto?

Práticas de projeto:

- **Diversificação:** identificar possíveis alternativas de projeto
- **Convergência:** rejeitar alternativas que não atendam os requisitos

O Modelo de Projeto

- A modelagem de projetos de software equivale aos planos de um arquiteto para uma casa

O Modelo de Projeto

- A modelagem de projetos de software equivale aos planos de um arquiteto para uma casa
- Começa pela representação do todo a ser construído (p. ex., uma representação tridimensional da casa)

O Modelo de Projeto

- A modelagem de projetos de software equivale aos planos de um arquiteto para uma casa
- Começa pela representação do todo a ser construído (p. ex., uma representação tridimensional da casa)
- Gradualmente, concentra-se nos detalhes, oferecendo um roteiro para a sua construção (p. ex., a estrutura do encanamento)

O Modelo de Projeto



Figura 1: Encanamento e tomadas da minha casa!

O Modelo de Projeto



Figura 2: Encanamento e tomadas da minha casa!

O Modelo de Projeto



Figura 3: Encanamento e tomadas da minha casa!

Exemplo: quais os principais módulos de um compilador?

O Modelo de Projeto

Exemplo: quais os principais módulos de um compilador?

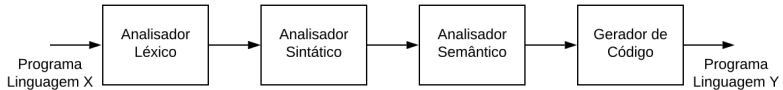


Figura 4: Principais módulos de um compilador.

O Modelo de Projeto

- Os elementos do modelo de projeto usam vários dos diagramas da UML utilizados no modelo de análise

O Modelo de Projeto

- Os elementos do modelo de projeto usam vários dos diagramas da UML utilizados no modelo de análise
- A diferença é o grau de refinamento, fornecendo detalhes para a implementação em relação à:
 - Estrutura e o estilo da arquitetura
 - Os componentes que residem nessa arquitetura
 - Interfaces entre os componentes e com o mundo exterior

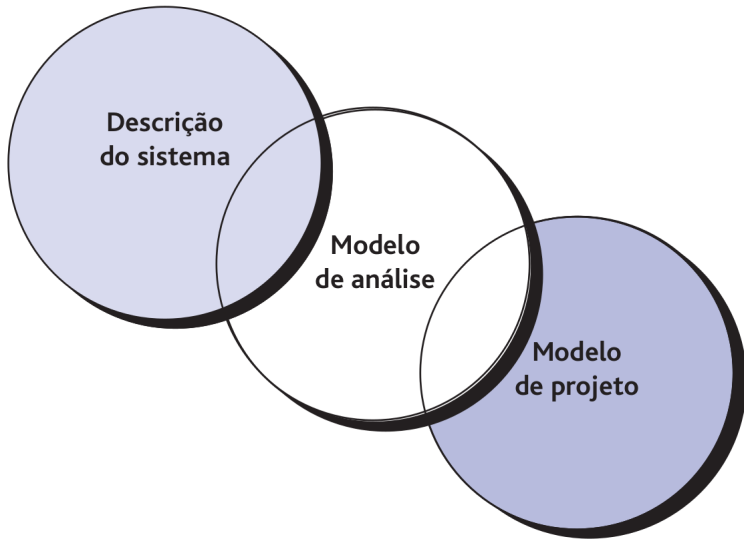


Figura 5: Descrição do sistema, modelo de análise e modelo de projeto.

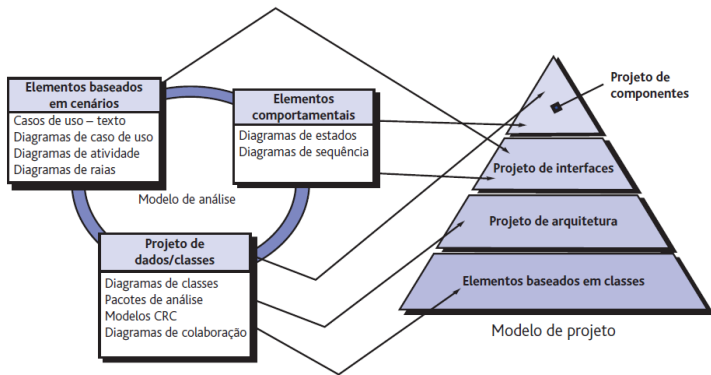


Figura 6: O modelo de projeto a partir do modelo de requisitos.

O Modelo de Projeto

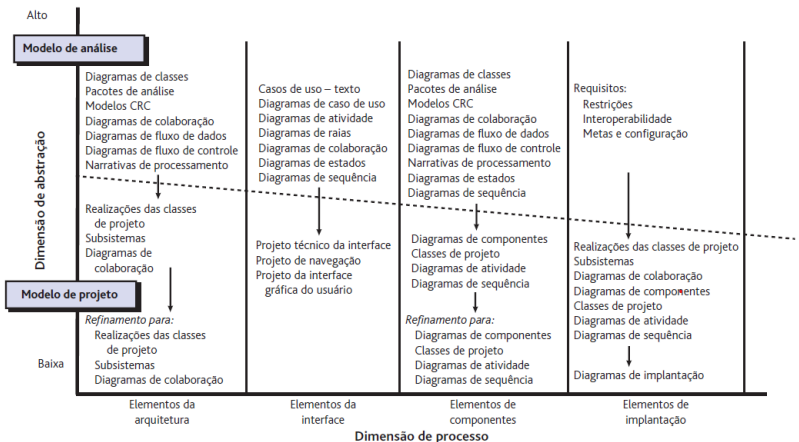


Figura 7: Dimensões do modelo de projeto.

Princípios da modelagem de projetos:

- Representações de projetos (modelos) devem ser de fácil compreensão

Princípios da modelagem de projetos:

- Representações de projetos (modelos) devem ser de fácil compreensão
- O projeto deve ser desenvolvido iterativamente

Princípios da modelagem de projetos:

- Representações de projetos (modelos) devem ser de fácil compreensão
- O projeto deve ser desenvolvido iterativamente
- A criação de um modelo de projeto não exclui uma abordagem ágil

Princípios da modelagem de projetos:

- Representações de projetos (modelos) devem ser de fácil compreensão
- O projeto deve ser desenvolvido iterativamente
- A criação de um modelo de projeto não exclui uma abordagem ágil
 - Alguns proponentes do desenvolvimento de software ágil dizem que o código é a única documentação de projeto necessária

Princípios da modelagem de projetos:

- Representações de projetos (modelos) devem ser de fácil compreensão
- O projeto deve ser desenvolvido iterativamente
- A criação de um modelo de projeto não exclui uma abordagem ágil
 - Alguns proponentes do desenvolvimento de software ágil dizem que o código é a única documentação de projeto necessária
 - Contudo, o objetivo de um modelo de projeto é ajudar outros que deverão manter e evoluir o sistema

Princípios da modelagem de projetos:

- Representações de projetos (modelos) devem ser de fácil compreensão
- O projeto deve ser desenvolvido iterativamente
- A criação de um modelo de projeto não exclui uma abordagem ágil
 - Alguns proponentes do desenvolvimento de software ágil dizem que o código é a única documentação de projeto necessária
 - Contudo, o objetivo de um modelo de projeto é ajudar outros que deverão manter e evoluir o sistema
 - Ao final do empreendimento, o projeto deve estar documentado em um nível que permita o entendimento e a manutenção do código

O processo de Projeto

O processo de projeto

- Processo iterativo:

O processo de projeto

- Processo iterativo:
 - Requisitos são traduzidos em uma “planta” para o desenvolvimento do software

O processo de projeto

- Processo iterativo:
 - Requisitos são traduzidos em uma “planta” para o desenvolvimento do software
 - Notação para representar componentes funcionais e suas interfaces

O processo de projeto

- Processo iterativo:
 - Requisitos são traduzidos em uma “planta” para o desenvolvimento do software
 - Notação para representar componentes funcionais e suas interfaces
 - Heurística para refinamento e divisões

O processo de projeto

- Processo iterativo:
 - Requisitos são traduzidos em uma “planta” para o desenvolvimento do software
 - Notação para representar componentes funcionais e suas interfaces
 - Heurística para refinamento e divisões
 - Diretrizes para avaliação da qualidade

O processo de projeto

- Processo iterativo:
 - Requisitos são traduzidos em uma “planta” para o desenvolvimento do software
 - Notação para representar componentes funcionais e suas interfaces
 - Heurística para refinamento e divisões
 - Diretrizes para avaliação da qualidade

O processo de projeto

- Deve implementar todos os requisitos explícitos do modelos de requisitos e tentar acomodar os requisitos implícitos

O processo de projeto

- Deve implementar todos os requisitos explícitos do modelos de requisitos e tentar acomodar os requisitos implícitos
- Deve ser um guia legível para os desenvolvedores de código, os que testam e dão suporte

O processo de projeto

- Deve implementar todos os requisitos explícitos do modelos de requisitos e tentar acomodar os requisitos implícitos
- Deve ser um guia legível para os desenvolvedores de código, os que testam e dão suporte
- Deve fornecer uma visão holística do software
 - Abstração de alto nível
 - Refinamentos subsequentes
 - Níveis menores de abstração

Tarefas genéricas para projeto

1. Examinar o modelo de informação e projetar estruturas de dados apropriadas para objetos de dados e seus atributos

Tarefas genéricas para projeto

1. Examinar o modelo de informação e projetar estruturas de dados apropriadas para objetos de dados e seus atributos
2. Usar o modelo de análise, selecionar um estilo de arquitetura (padrão) apropriado ao software

Tarefas genéricas para projeto

1. Examinar o modelo de informação e projetar estruturas de dados apropriadas para objetos de dados e seus atributos
2. Usar o modelo de análise, selecionar um estilo de arquitetura (padrão) apropriado ao software
3. Dividir o modelo de análise em subsistemas de projeto e alocá-los na arquitetura:
 - Certificar-se de que cada subsistema seja funcionalmente coeso
 - Projetar interfaces de subsistemas
 - Alocar classes ou funções de análise para cada subsistema

Tarefas genéricas para projeto

1. Examinar o modelo de informação e projetar estruturas de dados apropriadas para objetos de dados e seus atributos
2. Usar o modelo de análise, selecionar um estilo de arquitetura (padrão) apropriado ao software
3. Dividir o modelo de análise em subsistemas de projeto e alocá-los na arquitetura:
 - Certificar-se de que cada subsistema seja funcionalmente coeso
 - Projetar interfaces de subsistemas
 - Alocar classes ou funções de análise para cada subsistema
4. Criar um conjunto de classes ou componentes de projeto:
 - Transformar a descrição de classes de análise em uma classe de projeto
 - Verificar cada classe de projeto em relação aos critérios de projeto
 - Definir métodos e mensagens associadas a cada classe de projeto
 - Avaliar e selecionar padrões de projeto para uma classe ou um subsistema de projeto
 - Rever as classes de projeto e revisar quando necessário

Tarefas genéricas para projeto

5. Projetar qualquer interface necessária para sistemas ou dispositivos externos

Tarefas genéricas para projeto

5. Projetar qualquer interface necessária para sistemas ou dispositivos externos
6. Projetar a interface do usuário
 - Revisar os resultados da análise de tarefas
 - Especificar a sequência de ações baseando-se nos cenários de usuário
 - Criar um modelo comportamental da interface
 - Definir objetos de interface e mecanismos de controle
 - Rever o projeto de interfaces e revisar quando necessário

Tarefas genéricas para projeto

5. Projetar qualquer interface necessária para sistemas ou dispositivos externos
6. Projetar a interface do usuário
 - Revisar os resultados da análise de tarefas
 - Especificar a sequência de ações baseando-se nos cenários de usuário
 - Criar um modelo comportamental da interface
 - Definir objetos de interface e mecanismos de controle
 - Rever o projeto de interfaces e revisar quando necessário
7. Conduzir o projeto de componentes.
 - Especificar todos os algoritmos em um nível de abstração relativamente baixo
 - Refinar a interface de cada componente
 - Definir estruturas de dados dos componentes
 - Revisar cada componente e corrigir todos os erros descobertos

Tarefas genéricas para projeto

5. Projetar qualquer interface necessária para sistemas ou dispositivos externos
6. Projetar a interface do usuário
 - Revisar os resultados da análise de tarefas
 - Especificar a sequência de ações baseando-se nos cenários de usuário
 - Criar um modelo comportamental da interface
 - Definir objetos de interface e mecanismos de controle
 - Rever o projeto de interfaces e revisar quando necessário
7. Conduzir o projeto de componentes.
 - Especificar todos os algoritmos em um nível de abstração relativamente baixo
 - Refinar a interface de cada componente
 - Definir estruturas de dados dos componentes
 - Revisar cada componente e corrigir todos os erros descobertos
8. Desenvolver um modelo de implantação

Conceitos de projetos

- **Abstração procedural:** refere-se a uma sequência de instruções que possui uma função específica

- **Abstração procedural:** refere-se a uma sequência de instruções que possui uma função específica
- O nome indica sua função, mas detalhes são omitidos
 - Por exemplo, a palavra *abrir* para uma porta
 - Define uma sequência de etapas procedurais (caminhar, alcançar maçaneta, girar etc)

- **Abstração procedural:** refere-se a uma sequência de instruções que possui uma função específica
- O nome indica sua função, mas detalhes são omitidos
 - Por exemplo, a palavra *abrir* para uma porta
 - Define uma sequência de etapas procedurais (caminhar, alcançar maçaneta, girar etc)
- Diferentes níveis de abstração podem ser usados
 - Histórias de usuário

- **Abstração procedural:** refere-se a uma sequência de instruções que possui uma função específica
- O nome indica sua função, mas detalhes são omitidos
 - Por exemplo, a palavra *abrir* para uma porta
 - Define uma sequência de etapas procedurais (caminhar, alcançar maçaneta, girar etc)
- Diferentes níveis de abstração podem ser usados
 - Histórias de usuário
 - Casos de uso

- **Abstração procedural:** refere-se a uma sequência de instruções que possui uma função específica
- O nome indica sua função, mas detalhes são omitidos
 - Por exemplo, a palavra *abrir* para uma porta
 - Define uma sequência de etapas procedurais (caminhar, alcançar maçaneta, girar etc)
- Diferentes níveis de abstração podem ser usados
 - Histórias de usuário
 - Casos de uso
 - Diagramas de classes ou pseudocódigo

- Refere-se à organização geral do software

- Refere-se à organização geral do software
 - Organização de componentes de programa (módulos)

- Refere-se à organização geral do software
 - Organização de componentes de programa (módulos)
 - A maneira como esses componentes interagem

- Refere-se à organização geral do software
 - Organização de componentes de programa (módulos)
 - A maneira como esses componentes interagem
 - A estrutura de dados que são usados pelos componentes

- Refere-se à organização geral do software
 - Organização de componentes de programa (módulos)
 - A maneira como esses componentes interagem
 - A estrutura de dados que são usados pelos componentes

- Refere-se à organização geral do software
 - Organização de componentes de programa (módulos)
 - A maneira como esses componentes interagem
 - A estrutura de dados que são usados pelos componentes
- Em um sentido mais amplo, os componentes podem ser generalizados para representar os principais elementos de um sistema e suas interações

Como a arquitetura é vista em processos ágeis?

Como a arquitetura é vista em processos ágeis?

- Geralmente é aceito que um estágio inicial do processo de desenvolvimento ágil esteja focado na criação do projeto de arquitetura geral do sistema

Como a arquitetura é vista em processos ágeis?

- Geralmente é aceito que um estágio inicial do processo de desenvolvimento ágil esteja focado na criação do projeto de arquitetura geral do sistema
- O desenvolvimento incremental das arquiteturas normalmente não é bem sucedido

Como a arquitetura é vista em processos ágeis?

- Geralmente é aceito que um estágio inicial do processo de desenvolvimento ágil esteja focado na criação do projeto de arquitetura geral do sistema
- O desenvolvimento incremental das arquiteturas normalmente não é bem sucedido
- Refatorar componentes é fácil. Porém, refatorar a arquitetura pode implicar em ter que alterar a maior parte dos componentes

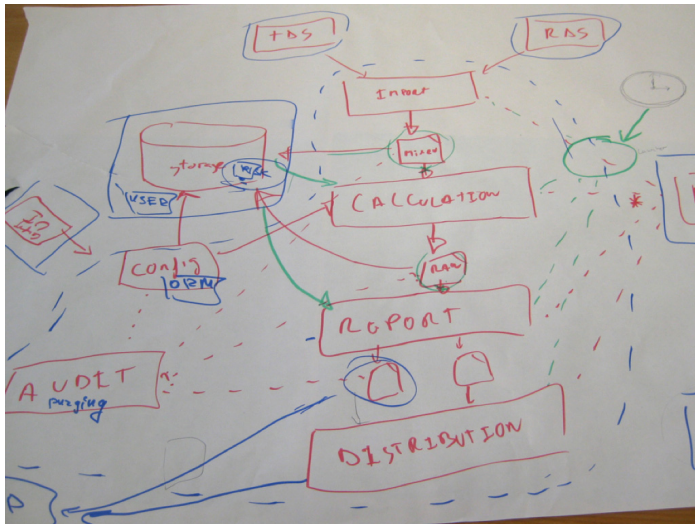


Figura 8: Rascunho de uma arquitetura de software.

O que é um padrão de projeto?

O que é um padrão de projeto?

Brad Appleton define da seguinte maneira:

“Padrão é parte de um conhecimento consolidado já existente que transmite a essência de uma solução comprovada para um problema recorrente em certo contexto, em meio a preocupações concorrentes.”

- O objetivo de um padrão é fornecer uma descrição que permita a um projetista determinar:
 - Se o padrão se aplica ou não ao problema em questão

- O objetivo de um padrão é fornecer uma descrição que permita a um projetista determinar:
 - Se o padrão se aplica ou não ao problema em questão
 - Se o padrão pode ou não ser reutilizado

- O objetivo de um padrão é fornecer uma descrição que permita a um projetista determinar:
 - Se o padrão se aplica ou não ao problema em questão
 - Se o padrão pode ou não ser reutilizado
 - Se o padrão pode servir como um guia para desenvolver um padrão similar, mas funcional ou estruturalmente diferente

Separação por interesses (afinidades)

- **Interesse:** característica, funcionalidade, comportamento ou requisito

Separação por interesses (afinidades)

- **Interesse:** característica, funcionalidade, comportamento ou requisito
- Sugere que qualquer problema complexo pode ser tratado mais facilmente se for subdividido em trechos a serem resolvidos e/ou otimizados independentemente

Separação por interesses (afinidades)

- **Interesse:** característica, funcionalidade, comportamento ou requisito
- Sugere que qualquer problema complexo pode ser tratado mais facilmente se for subdividido em trechos a serem resolvidos e/ou otimizados independentemente
- **Estratégia:** dividir-para-conquistar.

Separação por interesses (afinidades)

- A separação por interesses se manifesta em outros conceitos de projeto:
 - Modularidade
 - Independência funcional
 - Refinamento

“[...] modularidade é o único atributo de software que possibilita um programa ser intelectualmente gerenciável.”

Myers (1978)

“[...] modularidade é o único atributo de software que possibilita um programa ser intelectualmente gerenciável.”

Myers (1978)

- Componentes separadamente especificados e localizáveis

“[...] modularidade é o único atributo de software que possibilita um programa ser intelectualmente gerenciável.”

Myers (1978)

- Componentes separadamente especificados e localizáveis
- Podem ser denominados de “módulos” (ou não)
- São integrados para satisfazer os requisitos de um problema

Portanto, quanto mais módulos, melhor?

Modularidade

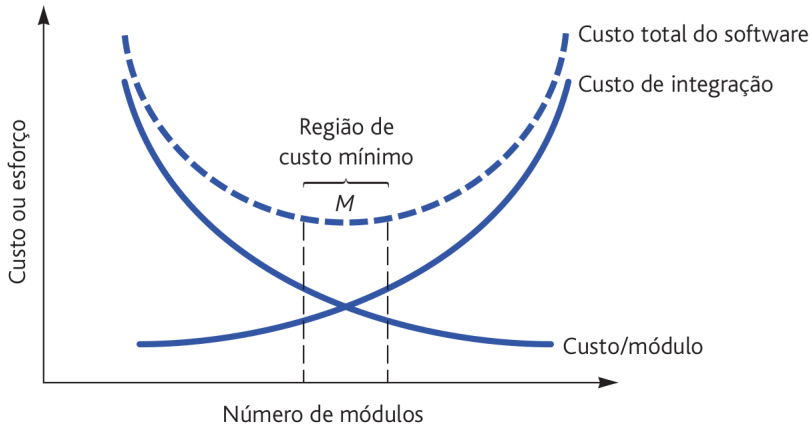


Figura 9: Relação entre o número de módulos e o esforço (custo) do software.

Como decompor uma solução de software para obter o melhor conjunto de módulos?

Isso nos leva ao conceito de **encapsulamento** de informações!

Isso nos leva ao conceito de **encapsulamento** de informações!

- **Modularidade efetiva:** obtida por meio de um conjunto de módulos independentes que comunicam entre si apenas as informações necessárias para realizar determinada função

Isso nos leva ao conceito de **encapsulamento** de informações!

- **Modularidade efetiva**: obtida por meio de um conjunto de módulos independentes que comunicam entre si apenas as informações necessárias para realizar determinada função
- O encapsulamento define e impõe **restrições de acesso**

Isso nos leva ao conceito de **encapsulamento** de informações!

- **Modularidade efetiva**: obtida por meio de um conjunto de módulos independentes que comunicam entre si apenas as informações necessárias para realizar determinada função
- O encapsulamento define e impõe **restrições de acesso**
- Informações (métodos e dados) contidas em um módulo devem ser **inacessíveis** por parte de outros módulos que não necessitam de tais informações

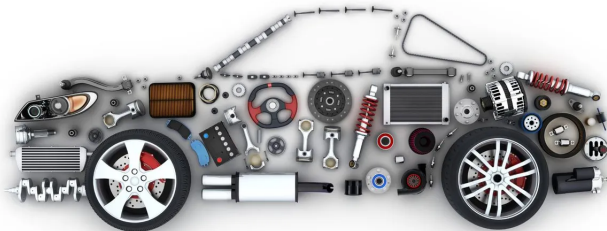


Figura 10: Partes do carro devem ser encapsuladas.

Encapsulamento

Em 2002, consta que Jeff Bezos enviou um mail para todos os desenvolvedores da empresa:

- Todos os times devem, daqui em diante, garantir que os sistemas exponham seus dados e funcionalidades por meio de interfaces

Encapsulamento

Em 2002, consta que Jeff Bezos enviou um mail para todos os desenvolvedores da empresa:

- Todos os times devem, daqui em diante, garantir que os sistemas exponham seus dados e funcionalidades por meio de interfaces
- Os sistemas devem se comunicar apenas por meio de interfaces

Encapsulamento

Em 2002, consta que Jeff Bezos enviou um mail para todos os desenvolvedores da empresa:

- Todos os times devem, daqui em diante, garantir que os sistemas exponham seus dados e funcionalidades por meio de interfaces
- Os sistemas devem se comunicar apenas por meio de interfaces
- Não deve haver outra forma de comunicação: sem links diretos, sem leitura direta em bases de dados de outros sistemas, sem memória compartilhada ou variáveis globais ou qualquer tipo de backdoor.

Encapsulamento

Em 2002, consta que Jeff Bezos enviou um mail para todos os desenvolvedores da empresa:

- Todos os times devem, daqui em diante, garantir que os sistemas exponham seus dados e funcionalidades por meio de interfaces
- Os sistemas devem se comunicar apenas por meio de interfaces
- Não deve haver outra forma de comunicação: sem links diretos, sem leitura direta em bases de dados de outros sistemas, sem memória compartilhada ou variáveis globais ou qualquer tipo de backdoor.
- Não importa qual tecnologia vocês vão usar: HTTP, CORBA, PubSub, protocolos específicos — isso não interessa. Bezos não liga para isso.

Encapsulamento

Em 2002, consta que Jeff Bezos enviou um mail para todos os desenvolvedores da empresa:

- Todos os times devem, daqui em diante, garantir que os sistemas exponham seus dados e funcionalidades por meio de interfaces
- Os sistemas devem se comunicar apenas por meio de interfaces
- Não deve haver outra forma de comunicação: sem links diretos, sem leitura direta em bases de dados de outros sistemas, sem memória compartilhada ou variáveis globais ou qualquer tipo de backdoor.
- Não importa qual tecnologia vocês vão usar: HTTP, CORBA, PubSub, protocolos específicos — isso não interessa. Bezos não liga para isso.
- Os times devem planejar e projetar interfaces pensando em usuários externos. Sem nenhuma exceção à regra.

Encapsulamento

Em 2002, consta que Jeff Bezos enviou um mail para todos os desenvolvedores da empresa:

- Todos os times devem, daqui em diante, garantir que os sistemas exponham seus dados e funcionalidades por meio de interfaces
- Os sistemas devem se comunicar apenas por meio de interfaces
- Não deve haver outra forma de comunicação: sem links diretos, sem leitura direta em bases de dados de outros sistemas, sem memória compartilhada ou variáveis globais ou qualquer tipo de backdoor.
- Não importa qual tecnologia vocês vão usar: HTTP, CORBA, PubSub, protocolos específicos — isso não interessa. Bezos não liga para isso.
- Os times devem planejar e projetar interfaces pensando em usuários externos. Sem nenhuma exceção à regra.
- Quem não seguir essas recomendações está demitido.

Encapsulamento

Em 2002, consta que Jeff Bezos enviou um mail para todos os desenvolvedores da empresa:

- Todos os times devem, daqui em diante, garantir que os sistemas exponham seus dados e funcionalidades por meio de interfaces
- Os sistemas devem se comunicar apenas por meio de interfaces
- Não deve haver outra forma de comunicação: sem links diretos, sem leitura direta em bases de dados de outros sistemas, sem memória compartilhada ou variáveis globais ou qualquer tipo de backdoor.
- Não importa qual tecnologia vocês vão usar: HTTP, CORBA, PubSub, protocolos específicos — isso não interessa. Bezos não liga para isso.
- Os times devem planejar e projetar interfaces pensando em usuários externos. Sem nenhuma exceção à regra.
- Quem não seguir essas recomendações está demitido.

Maiores benefícios do encapsulamento ocorrem quando são necessárias modificações durante os testes e, posteriormente, durante a manutenção do software!

Maiores benefícios do encapsulamento ocorrem quando são necessárias modificações durante os testes e, posteriormente, durante a manutenção do software!

Por quê?

Erros introduzidos inadvertidamente durante a modificação em um módulo têm menor probabilidade de se **propagar** para outros módulos ou locais dentro do software.

Independência funcional

- A independência funcional é atingida desenvolvendo-se módulos:

Independência funcional

- A independência funcional é atingida desenvolvendo-se módulos:
 - Com **função única**

Independência funcional

- A independência funcional é atingida desenvolvendo-se módulos:
 - Com **função única**
 - Com **aversão à interação excessiva** com outros módulos

Independência funcional

- A independência funcional é atingida desenvolvendo-se módulos:
 - Com **função única**
 - Com **aversão à interação excessiva** com outros módulos
- Por isso, a independência é avaliada por dois critérios qualitativos:

Independência funcional

- A independência funcional é atingida desenvolvendo-se módulos:
 - Com **função única**
 - Com **aversão à interação excessiva** com outros módulos
- Por isso, a independência é avaliada por dois critérios qualitativos:
 - **Coesão:** indica a robustez funcional relativa de um módulo, ou seja, tem apenas uma função

Independência funcional

- A independência funcional é atingida desenvolvendo-se módulos:
 - Com **função única**
 - Com **aversão à interação excessiva** com outros módulos
- Por isso, a independência é avaliada por dois critérios qualitativos:
 - **Coesão:** indica a robustez funcional relativa de um módulo, ou seja, tem apenas uma função
 - **Acoplamento:** indica a interdependência relativa entre os módulos

Coesão:

Coesão:

As histórias de usuário que contêm muitas instâncias de palavras como **e** ou **exceto** provavelmente não o incentivarão a projetar módulos com funções coesas

Coesão:

As histórias de usuário que contêm muitas instâncias de palavras como **e** ou **exceto** provavelmente não o incentivarão a projetar módulos com funções coesas

Exemplo para uma função com sério problema de coesão:

Independência funcional

Coesão:

As histórias de usuário que contêm muitas instâncias de palavras como **e** ou **exceto** provavelmente não o incentivarão a projetar módulos com funções coesas

Exemplo para uma função com sério problema de coesão:

```
float sin_or_cos(double x, int op) {  
    if (op == 1)  
        calcula e retorna seno de x  
    else  
        calcula e retorna cosseno de x  
}
```


Coesão:

Coesão:

Exemplo de classe coesa:

Coesão:

Exemplo de classe coesa:

```
class Stack<T> {  
    boolean empty() { ... }  
    T pop() { ... }  
    push (T) { ... }  
    int size() { ... }  
}
```

Independência funcional

Exemplo de acoplamento ruim entre classes:: um arquivo é compartilhado por duas classes, A e B, mantidas por desenvolvedores distintos. O método `B.g()` grava um inteiro no arquivo, que é lido por `A.f()`.

Independência funcional

Exemplo de acoplamento ruim entre classes:: um arquivo é compartilhado por duas classes, A e B, mantidas por desenvolvedores distintos. O método B.g() grava um inteiro no arquivo, que é lido por A.f().

```
class A {  
    private void f() {  
        int total; ...  
        File arq = File.open("arq1.  
            db");  
        total = arq.readInt();  
        ...  
    }  
}
```

```
class B {  
    private void g() {  
        int total;  
        // computa valor de total  
        File arq = File.open("arq1.  
            db");  
        arq.writeInt(total);  
        ...  
        arq.close();  
    }  
}
```

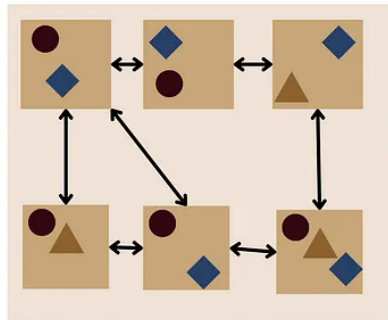
Uma solução melhor para o acoplamento::

Independência funcional

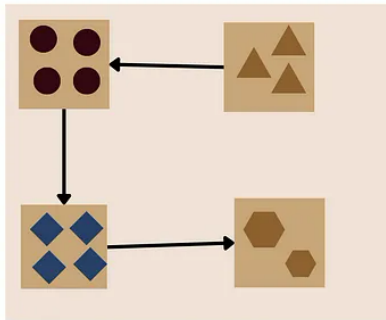
Uma solução melhor para o acoplamento::

```
class A {  
    private void f(B b) {  
        int total;  
        total = b.getTotal();  
        ...  
    }  
}
```

```
class B {  
    int total;  
  
    public int getTotal() {  
        return total;  
    }  
  
    private void g() {  
        // computa valor de total  
        File arq = File.open("arq1");  
        arq.writeInt(total);  
        ...  
    }  
}
```



Low cohesion and high coupling



High Cohesion and low Coupling

Figura 11: Relação entre coesão e acoplamento.

- É uma técnica de **reorganização** que simplifica o projeto (ou código) de um componente **sem mudar sua função**

- É uma técnica de **reorganização** que simplifica o projeto (ou código) de um componente **sem mudar sua função**
 - Melhora a estrutura interna sem modificar seu comportamento externo

- É uma técnica de **reorganização** que simplifica o projeto (ou código) de um componente **sem mudar sua função**
 - Melhora a estrutura interna sem modificar seu comportamento externo
 - Podem ocorrer efeitos colaterais involuntários

- É uma técnica de **reorganização** que simplifica o projeto (ou código) de um componente **sem mudar sua função**
 - Melhora a estrutura interna sem modificar seu comportamento externo
 - Podem ocorrer efeitos colaterais involuntários
 - Quais aspectos devem ser observados para refatoração?

- Para refatoração, o projeto é examinado em termos de:

- Para refatoração, o projeto é examinado em termos de:
 - Redundância (elementos de projeto não utilizados)

- Para refatoração, o projeto é examinado em termos de:
 - Redundância (elementos de projeto não utilizados)
 - Algoritmos ineficientes ou desnecessários

- Para refatoração, o projeto é examinado em termos de:
 - Redundância (elementos de projeto não utilizados)
 - Algoritmos ineficientes ou desnecessários
 - Estruturas de dados mal construídas ou inadequadas

- Para refatoração, o projeto é examinado em termos de:
 - Redundância (elementos de projeto não utilizados)
 - Algoritmos ineficientes ou desnecessários
 - Estruturas de dados mal construídas ou inadequadas
 - Qualquer outra deficiência ou falha de projeto que possa ser melhorada ou corrigida para produzir um projeto melhor

Refatoração preparatória:

Refatoração preparatória:

- O código não está estruturado para a adição de uma nova característica ou funcionalidade

Refatoração preparatória:

- O código não está estruturado para a adição de uma nova característica ou funcionalidade
- Portanto, deve-se, primeiro, refatorar o código para facilitar a adição da característica

Refatoração preparatória:

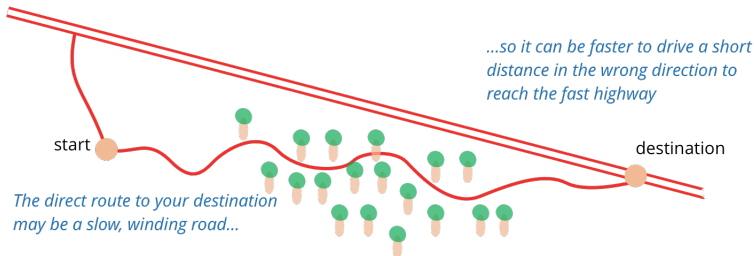


Figura 12: Refatoração preparatória.

Catálogo online de refatorações:

<https://www.refactoring.com/catalog/index.html>

- Com a evolução do modelo de projeto, define-se um conjunto de **classes de projeto** que **refina as classes de análise**

Classes de projeto

- Com a evolução do modelo de projeto, define-se um conjunto de **classes de projeto** que **refina as classes de análise**
- As **classes de análise** representam **objetos de dados e serviços** associados aplicados a eles

Classes de projeto

- Com a evolução do modelo de projeto, define-se um conjunto de **classes de projeto** que **refina as classes de análise**
- As **classes de análise** representam **objetos de dados e serviços** associados aplicados a eles
- Classes de projeto apresentam um **detalhe significativamente mais técnico** como um guia para a implementação

Referências

- [1] R. S. Pressman and B. R. Maxim.
Engenharia de Software uma abordagem profissional, volume 1.
AMGH Editora Ltda, 9. edition, 2021.
- [2] S. R. Schach.
Engenharia de Software: Os paradigmas clássicos & orientados a objetos, volume 1.
McGraw-Hill, 7. edition, 2008.
- [3] I. Sommerville.
Engenharia de Software, volume 1.
São Paulo: Addison-Wesley, 10. edition, 2019.
- [4] M. T. Valente.
Engenharia de software moderna, volume 1.
2020.

[5] R. S. Wazlawick.

Engenharia de Software - Conceitos e Prática, volume 1.

Rio de Janeiro: GEN LTC, 2. edition, 2019.

Perguntas?

Obrigado pela atenção!